



DLSU-D

DE LA SALLE UNIVERSITY-DASMARIÑAS

Introduction to Typescript

Author: Steven Villarosa



Vanilla Javascript

- Vanilla Javascript is a term that means a plain JS. No framework, no extension, but the core language itself.
- Is a dynamically typed language.
- A common problem with dynamically typed language is it's prone to:
 - Runtime error
 - Logical error



Sample 1 - Runtime error

sample1.js

```
const message = "Hello World!";  
message.toFixed(2);
```

- What is the error in the code above?
- How to fix this error?



index.html

```
<input id="input1" type="number" />
<input id="input2" type="number" />
<button id="add">Add</button>
<script src="sample2.js"></script>
```

Sample 2 - Logical error

sample2.js

```
document.querySelector("#add").addEventListener("click", () => {
  const userInput1 = document.querySelector("#input1");
  const userInput2 = document.querySelector("#input2");
  add(userInput1.value, userInput2.value);
});

function add(input1, input2) {
  console.log(input1 + input2);
}
```

- The data type returned by value is always string regardless of the input type.
- The add() function does not check the types of parameters.



The need for a solution

- In the previous examples, it is very intuitive as we can see that there is a change in the data type.
- However, in a large code base, it can be tough to determine what causes the error.
- Microsoft developed Typescript to solve this problem.

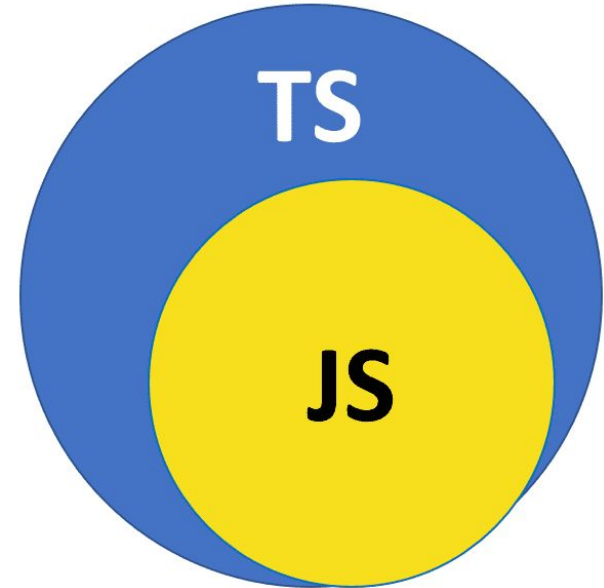


Typescript

- is a superset of Javascript
- adds syntax on top of JavaScript
- browsers can't execute typescript.

Gives additional value through the following:

- Type Safety
- Static Analysis(IDE Support)
- Other features such as:
 - Interfaces
 - Generics
 - Decorators



Node.js

- is an open source server environment
- allows you to run JavaScript on the server.

Npm

- was created as a package manager for Node.js.
- use to download software packages without any registration or logon.
- includes a CLI (Command Line Client) that can be used to download and install software:

```
C:\>npm install <package>
```



Install Typescript

To install typescript:

1. Install the latest version of [Node.js](#)
2. Open terminal(command prompt) and write the command:

```
npm install typescript -g
```

3. To check if TS is installed successfully, run the command

```
tsc -v
```



Typescript - Sample 1

sample1.ts

```
const message = "Hello World!";  
message.toFixed(2);
```

- Create the file above in a directory.
- Compare the behaviour of your IDE(VS Code) when the file is JS to its behavior when it is TS.
- In the same directory, run the command:
tsc sample1.ts

After running the tsc command, the corresponding JS file is created.

However, the runtime error is prevented since the IDE informs you of the error during static analysis.

sample1.js

```
const message = "Hello World!";  
message.toFixed(2);
```

The error now is that the IDE thought you redeclared the same variable. Ignore it for now.



Typescript - Sample 2

Below is the TS form of sample2.js. Copy and paste the code below into sample2.ts

sample2.ts

```
document.querySelector("#add")!.addEventListener("click", () => {  
    const userInput1 = document.querySelector("#input1") as HTMLInputElement;  
    const userInput2 = document.querySelector("#input2") as HTMLInputElement;  
    add(+userInput1.value, +userInput2.value);  
});  
  
function add(input1: number, input2: number) {  
    console.log(input1 + input2);  
}
```

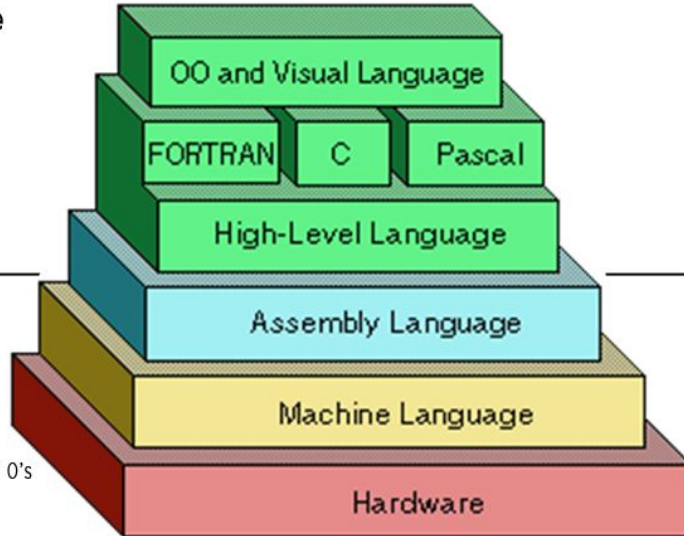
- Keep the html file as is.
- Run the command: **tsc sample2.ts**
- Observe what happened to the generated file sample2.js



High-Level vs Low-Level Programming Languages

High Level Language

- Easy for Programmers to understand
- Contains English Words



Low Level Language

- The computer's own Language
- Binary numbers, in 1's and 0's

High-level program

```
class Triangle {  
    ...  
    float surface()  
        return b*h/2;  
}
```

Low-level program

```
LOAD r1,b  
LOAD r2,h  
MUL r1,r2  
DIV r1,#2  
RET
```

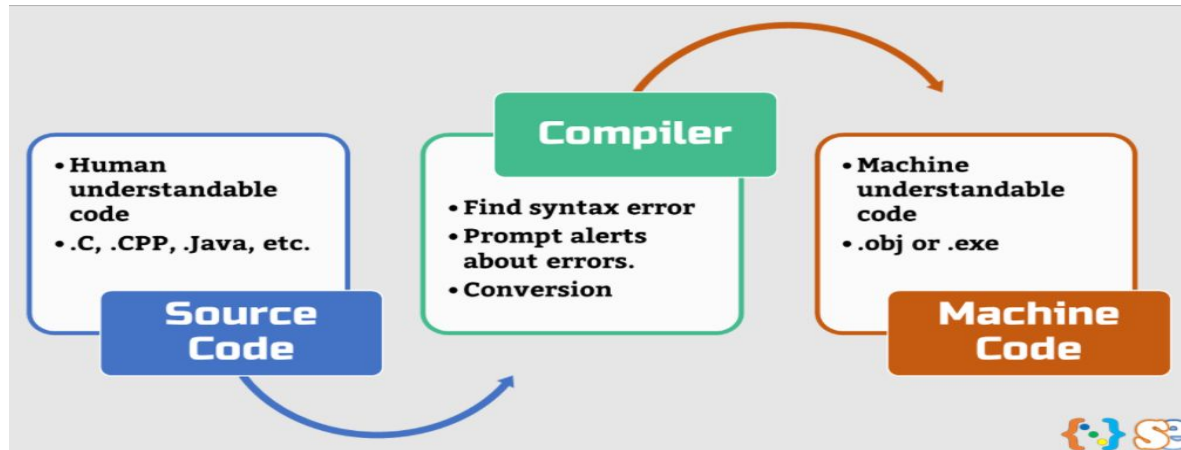
Executable Machine code

```
0001001001000101  
0010010011101100  
10101101001...
```



Compiler

- computers only understand binary language.
- compiler translates our source code in binary language to the computer can understand it.
- is a program which converts high level language to low level language.

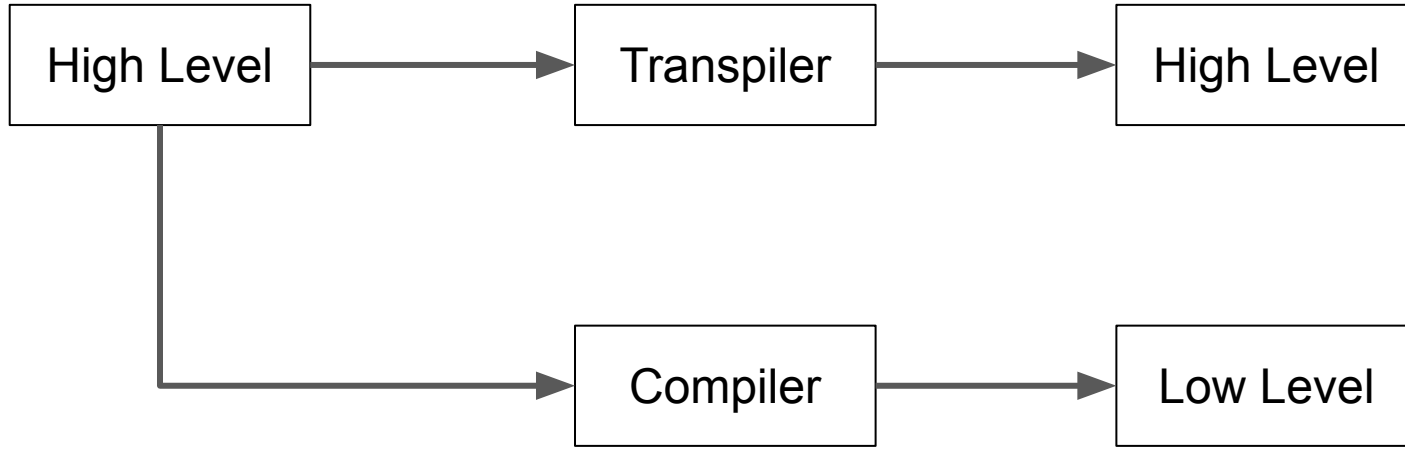


Transpiler

- also known as source-to-source compiler
- it takes source code written in one language and transform it into another language that has a similar level of abstraction
- transform a high level language to another high level language.
- Typescript is not a compiler because it translates typescript code, which is a high level language, into javascript, which is also a high level language.



Transpiler vs. Compiler



- Transpiler transform a high level language to another high level language.
- Compiler transform high level language to a low level language.



Initiating a Typescript Project

1. Create a directory. Let's name it "my_project".
2. Inside the directory, run the command: `npm init`.

Press enter to the all questions to set its default value. This should generate a file `package.json`. This contains your project configurations(esp. dependencies).

3. Add the initial files such as `index.html`, `app.js`, and `app.ts`.

Add the boiler plate template for html and point the script src to js not ts.

4. Install lite-server by running command: `npm install --save-dev lite-server`

Notice that the `package.json` has been added an entry called "devDependencies" with the property "lite-server": and its value is the lite-server version. You will also see a folder called "node_modules" this is where all your dependencies be stored.



Initiating a Typescript Project

5. Initiate a git repository by running the command: `git init`

You will notice that there will be thousands of files will be tracked. That's because of the `node_modules` directory containing all your project dependencies.

6. Create a file named `".gitignore"`.

In this file, list down all directories and files that you don't wish to be track by git. In this case, add `"node_modules"`. After saving, you should be able to see that the untracked files are reduced.

7. Add the start command in the `package.json` file.

In the `"scripts"` property, add the value `"start": "lite-server"`. See the sample [package.json](#) for your reference.



Initiating a Typescript Project

8. Run the start command: `npm start`

This will serve your file `index.html` in your browser using the default address `http://localhost:3000/`. It watches all your files

9. Open another terminal to compile your ts file: `tsc app.ts -w`
10. Connect your local repo to your remote repo and push: `git remote add`
Use this [slide](#) for your reference. Once you pushed your changes, you should notice that all only the necessary files are pushed. The "node_modules" folder is excluded.
11. Optional. If you are in another environment, try to clone the repository and install the dependencies by using: `npm install` in the folder directory.



Core Types

- number
- string
- boolean
- object
- array
- tuple
- enum
- any - Never use this type
- Function
- unknown - Never use this type



Type Inference

Typescript automatically assign a type to a variable when it is assigned during initialization. Ex:

```
let n = 5;
```

Typescript automatically set the type of n to number because the initial value is set to 5. When you try to assign a string value to n:

```
n = "hello";
```

TS will throw a compilation error "Type 'string' is not assignable to type 'number'."



Type Inference

Type inference is also applied in function return type.

```
function getNumber() {  
    return 4;  
}
```

When you hover the function name, the editor should display something like
'function hello(): number'

We can also explicitly declare the return type of a function using : after the function parameters.

```
function getNumber(): number {  
    return 4;  
}
```



Union Types

Allows a variable to have multiple types. Ex:

```
let result : number | string;  
result = 5;  
result = "Hello World!";
```

It can also be used to function parameter and return type:

```
function fn1(param1: number | string) {...}  
function fn2(): boolean | string {...}  
function getObject(): { [k: string]: number | string } {...}
```



Literal Types

- are fixed values that can be assigned to a variable
`const num: 5 = 5;`
- literal types are not that useful on their own but can be used in conjunction with union types to describe a finite set of possible values

```
function getDirection(): "n" | "s" | "e" | "w" {...}
```

```
function printMyNumber(myNumber: 5 | 6 | 8) {...}
```

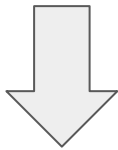
```
function getUnit(system: "metric" | "english"): "m" | "ft"
{
    if (system === "metric") return "m";
    return "ft";
}
```



Type Aliases

When using union types in multiple places, we can create type aliases to make it easier and prevent future bugs.

```
function getType(val: string | number | boolean | null) {}  
function guestType(): string | number | boolean | null {}
```



```
type Primitive = string | number | boolean | null;  
function getType(val: primitive) {}  
function guestType(): Primitive {}
```



Interface

- describes the structure of an object.
- like a custom object type.
- the variables inside the interface cannot have a default value.
- the functions inside the interface cannot have an implementation.

```
interface Person {  
  name: string;  
  age: number;  
  greet(phrase: string): void;  
}
```

```
let user1: Person;  
user1 = {  
  name: "Steven",  
  age: 30,  
  greet(ph: string) {  
    console.log(`${ph} ${this.name}`);  
  },  
};
```

```
user1.greet("Hi there - I am");
```



Interface vs Type

- both interface and type can be use to structure an object

```
type Student = {  
  student_id: number;  
  name: string;  
  birth_day?: Date;  
};
```

```
let student1: Student = {  
  student_id: 1,  
  name: "Steven",  
};  
console.log(student1.name);
```

```
interface Subject {  
  subject_id: string;  
  name: string;  
  units?: number;  
}
```

```
let subject1: Subject = {  
  subject_id: "COMP20133",  
  name: "App Development",  
};  
console.log(subject1.name);
```



Interface vs Type

- however, type can be use to create union types while interface can be implemented in an object.

```
type Primitive = string | number | boolean | null;
```

```
interface Shape {  
  color: string;  
  draw(): void;  
}
```

```
class Circle implements Shape {  
  color = "red";  
  
  draw(): void {  
    console.log("draw circle");  
  }  
}
```



Enum

- allow a developer to define a set of named constants.
- by default, enum values are numeric and starts in 0.

```
enum Direction {  
    UP,  
    DOWN,  
    LEFT,  
    RIGHT,  
}  
console.log(Direction.UP);
```

- however, each element in the enum can be assigned a custom value of any type.



Enum

- help to prevent magic values.
- make sure that related constants are grouped together.
- to narrow down the values that can be assigned to a variable.

```
enum DistanceUnit {  
    METER = "m",  
    KM = "km",  
    MILE = "mi",  
    FOOT = "ft",  
}
```

```
let myUnit: DistanceUnit;  
myUnit = DistanceUnit.FOOT;  
myUnit = DistanceUnit.KM;  
myUnit = "ft";
```



Guidelines

- Always configure .gitignore to ignore node_modules directory.
- Always specify the type for every function parameters.
- Always specify the return type of the function.
- Use enum to group together related constants.
- Enum name should be in Pascal case. Enum elements should be capitalized snake case.



Guidelines

- Use Pascal case for type aliases and classes



```
type Primitive = string | number | boolean |  
null;
```



```
type primitive = string | number | boolean |  
null;
```



End of Discussion



package.json

```
{
  "name": "my_project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "lite-server"
  },
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "lite-server": "^2.6.1"
  }
}
```



References:

https://www.w3schools.com/whatis/whatis_npm.asp

https://www.w3schools.com/nodejs/nodejs_intro.asp

https://www.w3schools.com/typescript/typescript_intro.php

<https://medium.com/@chaewonkong/typescript-what-good-for-8240dc9173c7>

<https://mohasinhaque23121.medium.com/compiler-vs-transpiler-a64c989607d7>

https://www.w3schools.com/typescript/typescript_aliases_and_interfaces.php

<https://www.typescriptlang.org/docs/handbook/enums.html>

